

SC2207 Introduction to Databases
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

- 1 Data Models, ER Diagrams, and ER-to-Relational Mapping 1**
 - 1.1 Why a Database? 1
 - 1.1.1 Levels of Data Abstraction 2
 - 1.1.2 Data Models—A Taxonomy 2
 - 1.2 The Entity–Relationship Model 3
 - 1.2.1 Attributes, Keys, and Domains 3
 - 1.2.2 Relationships, Cardinality, Participation 3
 - 1.3 From ER to Relational Schema 3
 - 1.3.1 Keys in the Relational Target 4
 - 1.3.2 Design Pitfalls to Avoid 4
 - 1.4 Chapter Notes 5
 - 1.5 Exercises 5
- 2 The Relational Model and Relational Algebra 6**
 - 2.1 From Sets to Relations 6
 - 2.2 Relational Algebra—Core Operators 7
 - 2.2.1 The Six Fundamentals 7
 - 2.2.2 Derived Operators: Joins and Division 8
 - 2.2.3 Expression Trees and Equivalences 8
 - 2.3 Integrity Constraints Formalised 9
 - 2.4 Chapter Notes 9
 - 2.5 Exercises 9
- 3 SQL: Definition, Manipulation, and Querying 10**

3.1	DDL: Creating the Schema	10
3.2	DML: Inserting, Updating, Deleting	11
3.3	SELECT: The Query Workhorse	11
3.3.1	Filtering and Projection	12
3.3.2	JOINS	12
3.3.3	Aggregation: GROUP BY and HAVING	13
3.3.4	Subqueries and Set Operations	13
3.4	Views	13
3.5	Programmatic Access (JDBC) and Authorization	14
3.6	Chapter Notes	14
3.7	Exercises	14
4	Relational Design and Normalisation	16
4.1	Good vs. Bad Design	16
4.2	Functional Dependencies	17
4.2.1	Closure and Armstrong’s Axioms	17
4.2.2	Keys from FDs	17
4.3	Normal Forms	18
4.4	Decomposition: Lossless and Dependency-Preserving	18
4.4.1	3NF Synthesis Algorithm	19
4.4.2	BCNF Decomposition Algorithm	19
4.4.3	Putting It Together: A Mini-Case	19
4.5	Chapter Notes	20
4.6	Exercises	20
5	Storage, Files, and Index Structures	21
5.1	The Memory Hierarchy and Disk	21
5.1.1	Pages, Records, and Buffer Manager	22
5.2	File Organisations	22
5.3	B ⁺ Trees—The Default Index	23
5.3.1	Clustered vs. Unclustered, Dense vs. Sparse	23

5.4	Hash Indexes	23
5.4.1	Example: Index Selection for NTU Schema	24
5.5	Chapter Notes	24
5.6	Exercises	24
6	Query Processing, Optimisation, and Cost Estimation	26
6.1	From Declarative Query to Execution Plan	26
6.1.1	Logical vs. Physical Operators	27
6.2	Physical Operators and Their Costs	27
6.2.1	Other Physical Choices	28
6.2.2	Pipelining vs. Materialisation	28
6.3	Cardinality Estimation and Histograms	28
6.4	Plan Enumeration and EXPLAIN	29
6.4.1	Heuristic Rewriting First	29
6.5	Chapter Notes	30
6.6	Exercises	30
7	Transactions, Concurrency Control, and Recovery	31
7.1	Transactions and ACID	31
7.2	Why Concurrency Is Hard: Anomalies	32
7.2.1	Precedence (Serialisation) Graph	32
7.3	Locking and Two-Phase Locking (2PL)	33
7.3.1	Deadlock	33
7.3.2	Beyond Locking: MVCC and Optimistic	33
7.4	Recovery: WAL and Checkpoints	34
7.5	Chapter Notes	34
7.6	Exercises	34
8	Advanced Topics: NoSQL, NewSQL, and Distributed Databases	36
8.1	Beyond Relations: Why NoSQL?	36
8.1.1	The Four NoSQL Families	37

8.1.2	When to Stay Relational?	37
8.2	CAP, PACELC, and Consistency Models	38
8.3	Replication and Quorums	38
8.4	Sharding (Partitioning)	39
8.4.1	Example: Sharding NTU Enrolled	39
8.5	NewSQL: ACID at Scale	39
8.6	Chapter Notes	40
8.7	Exercises	40

Chapter 1

Data Models, ER Diagrams, and ER-to-Relational Mapping

“Data knows the truth—if you model it faithfully.” Before writing a single SQL statement we must decide what entities exist, how they relate, and what constraints govern them. This chapter builds that conceptual foundation from first principles.

From zero: no database assumed. We start from everyday records (a student list, a library) and build every notion—entity, attribute, relationship, key, abstraction level—from pictures and intuition before any formalism. No prior database course is needed. Later chapters depend only on what is built here.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish physical, logical, and conceptual abstraction levels;
- (L2) define and draw Entity–Relationship diagrams with cardinality and participation;
- (L3) Identify keys, weak entities, and relationship attributes correctly;
- (L4) map any ER diagram to a relational schema using the standard rules;
- (L5) critique a design for redundancy and constraint loss.

1.1 Why a Database?

Consider keeping student grades in a spreadsheet shared by email. Soon copies diverge, two tutors overwrite the same cell, and nobody knows which file is authoritative. A file system stores bytes; a *database management system* (DBMS) adds structure, integrity, concurrency, durability, and a declarative query language on top.

Definition 1.1 (Database and DBMS). A *database* is a structured collection of related data modelling some enterprise. A *DBMS* is software that stores, queries, and manages that data while enforcing consistency and supporting concurrent access.

Think of the DBMS as a disciplined librarian: it catalogues every item, enforces who may borrow what, resolves conflicts when two people request the same book, and guarantees no record is lost even if the lights go out.

Key advantages over flat files: *data independence* (change storage without rewriting apps), *declarative access* (say what you want, not how to fetch it), *transactional guarantees* (Ch. 7), and *centralised control* (security, backup, schema evolution).

1.1.1 Levels of Data Abstraction

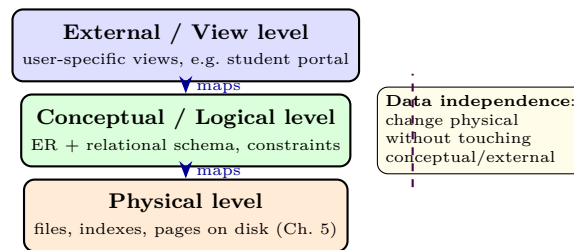


Figure 1.1: Three abstraction levels. Views shield users; conceptual schema is the contract; physical layer optimises storage.

Like architectural drawings: the owner’s sketch (view), the architect’s blueprint (conceptual), and the engineer’s construction detail (physical). The blueprint insulates the two.

1.1.2 Data Models—A Taxonomy

A *data model* is the language in which a design is expressed.

- **Hierarchical / Network** (1960s–70s)—trees or graphs of records navigated via pointers (IMS, CODASYL). Fast but rigid; program coupled to traversal path.
- **Relational** (Codd, 1970)—data as tables (relations); queried declaratively by algebra/SQL. Dominant for OLTP.
- **Entity–Relationship**—conceptual picture of entities and relationships; not implemented directly but mapped to relational.
- **Semi-structured / Object models**—JSON, XML, objects (Ch. 8) for flexible or graph-shaped data.

This book takes ER → relational → SQL → internals. Start with ER because it thinks in enterprise concepts, not tables.

1.2 The Entity–Relationship Model

An *entity* is a distinguishable object (a particular student). An *entity set* is a collection of similar entities (all students). Each entity has *attributes* (properties like ID, name).

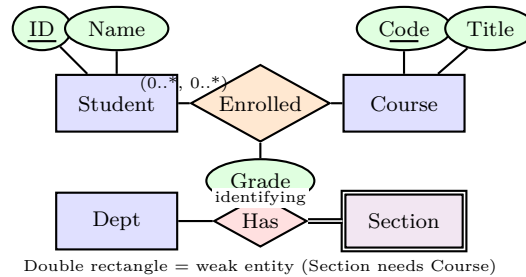


Figure 1.2: ER elements: rectangles = entity sets, diamonds = relationships, ovals = attributes, underlining = key; double diamond/rectangle = identifying relationship / weak entity. Grade is a relationship attribute.

1.2.1 Attributes, Keys, and Domains

Each attribute has a *domain* (allowed values). Attribute types: *simple* vs. *composite* (Address $\rightarrow$ Street/City), *single-valued* vs. *multi-valued* (double oval), *derived* (dashed oval, e.g. age from DOB).

A *superkey* uniquely identifies entities; a *key* is a minimal superkey; a *candidate key* is any minimal key, one chosen as *primary key* (underlined). Weak entities (e.g. Section identified only via Course) use a *discriminator* (partial key) plus the owner’s key.

1.2.2 Relationships, Cardinality, Participation

A *relationship set* associates entities. *Degree*: binary (most common), ternary, etc. Each entity set’s participation has:

- **Cardinality ratio**—1:1, 1:N, N:1, M:N (how many of the other side one entity may relate to). Example: a Course *has* many Sections (1:N).
- **Participation constraint**—*total* (double line: every entity must participate) vs. *partial* (single line: optional).

An arrow to an entity set means *at most one*; a double line means *at least one*. So a thick arrow-headed double line = exactly one.

Remark 1.1 (Attributes on relationships). Place an attribute on a relationship when it describes the association, not either entity alone (e.g. Grade belongs to Enrolled, not to Student or Course). If in doubt, ask: “Would this value exist without the relationship?”

1.3 From ER to Relational Schema

Mapping is mechanical once constraints are understood.

1. **Strong entity set** $\rightarrow$ table with same attributes; primary key as declared.
2. **Weak entity set** $\rightarrow$ table with owner's PK + discriminator; PK is composite.
3. **Binary 1:N** $\rightarrow$ add the “one” side's PK as foreign key to the “many” side (no separate table). Place relationship attributes on the many side.
4. **Binary M:N** $\rightarrow$ new join table with PKs of both sides (composite PK); relationship attributes go here.
5. **Binary 1:1** $\rightarrow$ either side may hold the other's PK (choose the total-participation side to avoid nulls); or a separate table if both partial.
6. **Multi-valued attribute** $\rightarrow$ separate table (PK of entity + value).
7. **N-ary relationship** $\rightarrow$ new table with FKs to all participating entity sets.

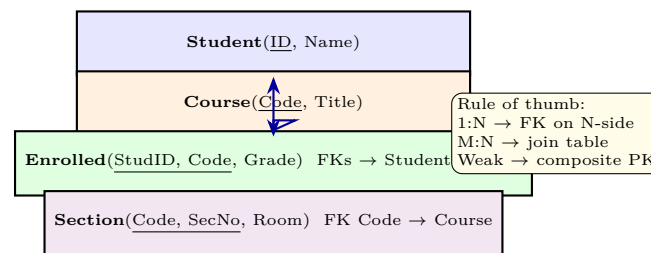


Figure 1.3: Relational schema resulting from Figure 1.2. Underline = primary key; FKs preserve relationships; null avoidance guides FK placement.

Example 1.1 (Worked mapping). ER: Student(M)—Enrolled(N)—Course, plus Course 1:N Section (weak). Relational: Student, Course as above; Enrolled join table captures M:N + Grade; Section reuses Course.Code. Cardinalities preserved via keys and foreign-key constraints; total participation becomes NOT NULL on FK.

Remark 1.2 (What can be lost?). ER participation (“every student must enrol in at least one course”) is not enforceable by keys alone—it needs assertions or triggers beyond basic DDL. Flag such constraints when mapping.

1.3.1 Keys in the Relational Target

After mapping, keys become concrete constraints. A *primary key* (PK) is a minimal set of columns whose values are unique and non-null for every row—exactly the ER key now enforced by the DBMS. A *foreign key* (FK) is a column set whose values must appear as a PK in the referenced table (referential integrity). An FK may be NOT NULL to enforce total participation, or nullable to allow partial participation. This is how the conceptual cardinality survives into tables.

Example 1.2 (Keys survive mapping). In Figure 1.3, Enrolled's composite PK (*StudID*, *Code*) prevents a student enrolling twice in the same course. Each column is also an FK, so you cannot enrol a non-existent student. If Course—Section is 1:N total on the Section side, Section.Code is NOT NULL FK.

1.3.2 Design Pitfalls to Avoid

- **Redundancy**—storing Course.Title inside every Enrolled row. Update one copy and miss another: inconsistency.

- **Overloaded keys**—using Name as key when names collide; always prefer stable surrogate or matrix number.
- **Misplaced attributes**—putting building capacity on Room when it belongs to Building.

A quick sanity check: can you state each attribute as “X’s Y” with X being the entity/relationship that owns it? If not, ownership is likely wrong.

1.4 Chapter Notes

The ER model is due to Peter Chen (1976); the abstraction-level view follows ANSI/SPARC (1975). Relational mapping rules are those of Elmasri–Navathe and Silberschatz et al. Modern practice often models first in ER/UML, then generates DDL automatically.

1.5 Exercises

- E1.1 **Warm-up: abstraction.** Give a concrete view, conceptual, and physical example for an NTU course-registration system. Why does changing the physical index not require rewriting the student view?
- E1.2 **ER drawing.** Model NTU’s “Student enrolls in Course Offerings taught by Instructor in a Room at a Timeslot.” Identify entities, keys, the ternary relationship, and where Grade and Capacity belong. Mark cardinalities.
- E1.3 **Weak entities.** Why is Room not weak but Section is? Design the tables for a weak entity Building → Room (RoomNo only unique within Building) and state its primary key.
- E1.4 **Mapping choices.** An ER says Every Course has exactly one Coordinator (Lecturer) but a Lecturer may coordinate zero or one Course. Show two relational mappings (FK on Course vs. FK on Lecturer vs. separate table) and argue which minimises nulls.
- E1.5 **Critique.** A novice puts StudentPhone as an attribute of Student (single-valued). Give an instance where this loses information, then redesign using a separate table. Which rule justifies it?

Chapter 2

The Relational Model and Relational Algebra

“Tables all the way down.” Codd’s relational insight was to represent everything—data, schemas, constraints, query results—as tables manipulated by a tiny, composable algebra. This chapter makes that world precise.

From zero: sets to schemas. We assume only the set notion from Ch. 1 and build relations, tuples, keys, and the algebra step by step with pictures before notation. Familiarity with ER is helpful but not required.

Learning Outcomes

After this chapter you will be able to:

- (L1) define relations, schemas, tuples, domains, and keys formally;
- (L2) explain integrity constraints (entity, referential, domain);
- (L3) write queries in relational algebra using the six core operators;
- (L4) translate between intuitive table operations and algebraic expressions;
- (L5) reason about equivalence and minimality of algebra expressions.

2.1 From Sets to Relations

A *domain* D is a set of atomic values (integers, strings, dates). An *attribute* A has a domain $\text{dom}(A)$.

Definition 2.1 (Relation, tuple, schema). Let attributes $A_1, \dots, A_n$ have domains $D_1, \dots, D_n$. A *relation* r of schema $R(A_1, \dots, A_n)$ is a finite *set* of tuples, each tuple t being a mapping $t[A_i] \in D_i$. We write $r(R)$ or $r \subseteq D_1 \times \dots \times D_n$. Degree = n ; cardinality = $|r|$.

Intuition: a table with named columns and no duplicate rows, where row order and column order (by name) do not matter. The schema is the header; the *instance* is the current set of rows. This differs from a spreadsheet: sets have no order and no duplicates by definition.

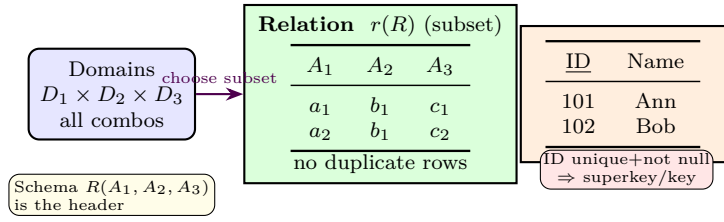


Figure 2.1: Left: a relation is a finite set of tuples drawn from the Cartesian product of domains. Right: a key enforces uniqueness.

Definition 2.2 (Keys and constraints). A set $K \subseteq R$ is a *superkey* if no two distinct tuples agree on all attributes of K . A *candidate key* is a minimal superkey; one is chosen as *primary key* (underlined). $\emptyset$ cannot be a key. Domain, entity (PK not null), and referential (FK $\rightarrow$ PK) constraints are the relational integrity rules.

Example 2.1 (Nulls vs. absence). SQL’s NULL is not a domain value; it means “unknown/absent.” In pure relational theory there are no nulls—every tuple provides a value for every attribute. SQL relaxes this but keeps PK columns non-null.

We also distinguish *schema* (intension, the definition) from *instance* (extension, the current rows). Queries map instances to instances; constraints restrict which instances are legal.

2.2 Relational Algebra—Core Operators

Algebra expressions compose operators on relations and yield relations (closure). *Selection* filters rows; *projection* filters columns; the rest combine relations.

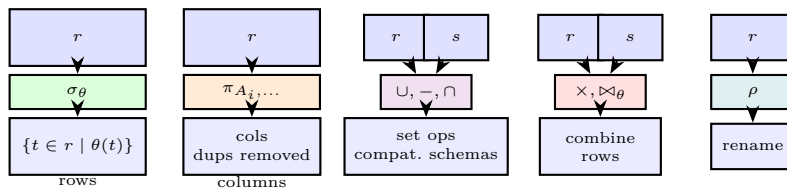


Figure 2.2: Core algebra operators. σ filters rows, π filters columns (and de-duplicates), set ops require union-compatible schemas, $\times/\bowtie$ combine.

2.2.1 The Six Fundamentals

1. Selection $\sigma_\theta(r)$. Keep tuples satisfying predicate θ (e.g. *Age* $>$ 21 and *Major* = ‘CS’). Example: $\sigma_{Grade \geq 80}(Enrolled)$.

2. Projection $\pi_{A_{i_1}, \dots, A_{i_k}}(r)$. Keep only those columns, then de-duplicate (set semantics). $\pi_{Name}(Student)$ lists names once.

3. Union $r \cup s$, **Difference** $r - s$, **Intersection** $r \cap s = r - (r - s)$. Require same arity and compatible domains (union compatibility). $r \cup s$ = tuples in either; $r - s$ = in r but not s .

4. Cartesian product $r \times s$. Every r -tuple paired with every s -tuple (degree sums, cardinality multiplies). Usually followed by selection (which is exactly a join).

5. Rename $\rho_{S(B_1, \dots, B_n)}(r)$ or $\rho_S(r)$. Rename relation and/or attributes to resolve clashes (e.g. self-joins). Essential for formal correctness.

These six are complete: any other operator can be derived, but we add convenient shorthands.

2.2.2 Derived Operators: Joins and Division

Definition 2.3 (Joins). $r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$ (theta-join). When θ is equality on common attributes, it is an equi-join; if the equality attribute appears once, it is a *natural join* $r \bowtie s$. Left outer join $r \text{--}\bowtie s$ keeps unmatched r tuples padded with nulls (similarly right/full).

Example 2.2 (Natural join vs. product). $\text{Student} \bowtie \text{Enrolled}$ matches on $\text{Student.ID} = \text{Enrolled.StudID}$ automatically, yielding one row per enrolment with student details attached. $\text{Student} \times \text{Enrolled}$ would pair every student with every enrolment indiscriminately—usually not what we want.

Division $r \div s$. For $r(A, B)$ and $s(B)$, $r \div s = \{a \mid \forall b \in s, (a, b) \in r\}$. Classic “for all” query: students who enrolled in *all* courses in s . Formally $r \div s = \pi_A(r) - \pi_A((\pi_A(r) \times s) - r)$.

2.2.3 Expression Trees and Equivalences

Algebra expressions are trees; evaluating bottom-up yields the answer. Two expressions are *equivalent* if they give the same result on every instance (e.g. $\sigma_p(\sigma_q(r)) \equiv \sigma_{p \wedge q}(r)$, $\sigma_p(r \bowtie s) \equiv \sigma_p(r) \bowtie s$ if p mentions only r). Optimisers (Ch. 6) use such laws to push selections down and reduce intermediate size.

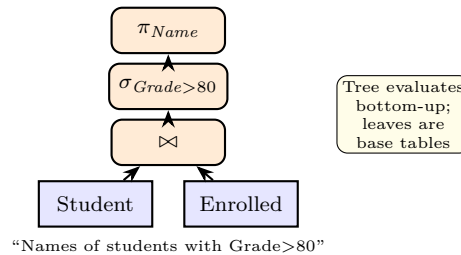


Figure 2.3: Algebra expression as a tree. Pushing σ below $\bowtie$ when possible shrinks intermediate results.

Useful laws: cascade $\sigma_p(\sigma_q(r)) \equiv \sigma_{p \wedge q}(r)$; commute $\sigma_p(\pi_A(r)) \equiv \pi_A(\sigma_p(r))$ if p mentions only A ; distribute $\pi_A(r \cup s) \equiv \pi_A(r) \cup \pi_A(s)$ but $\pi_A(r - s) \not\equiv \pi_A(r) - \pi_A(s)$ in general.

2.3 Integrity Constraints Formalised

- **Domain** $t[A] \in \text{dom}(A)$ plus **CHECK** predicates.
- **Entity** Primary key attributes not null and unique: if $t_1[K] = t_2[K]$ then $t_1 = t_2$.
- **Referential** For FK F referencing PK K of s : $\pi_F(r) \subseteq \pi_K(s) \cup \{\text{null}\}$ (or without null if **NOT NULL**).

Violations arise on insert/update/delete. DBMS options: reject, **CASCADE**, **SET NULL**, **SET DEFAULT**—declarative policies attached to the FK.

Remark 2.1 (Why sets?). Set semantics makes algebra simple and duplicate-free, matching Ch. 4’s normalisation theory. SQL uses multisets (bags) for performance; **DISTINCT** recovers set behaviour when needed.

2.4 Chapter Notes

Codd’s 1970 paper “A Relational Model of Data for Large Shared Data Banks” founded the field. Our operator set follows Silberschatz–Korth–Sudarshan and Ullman–Widom. Outer joins and division are syntactic sugar but pedagogically essential. Bag semantics vs. set semantics tension resurfaces in SQL (Ch. 3) and optimisation (Ch. 6).

2.5 Exercises

- E2.1 **Warm-up: schemas.** Given $R(A, B, C)$ with 2+3+2 distinct values per domain, what are $\max |r|$ and degree? Give an instance where A is a key but B is not.
- E2.2 **Core algebra.** Using $\text{Student}(\text{ID}, \text{Name}, \text{Major})$ and $\text{Enrolled}(\text{StudID}, \text{Code}, \text{Grade})$, write algebra for: (a) CS majors, (b) names of students with $\text{Grade} > 85$, (c) students enrolled in no course, (d) courses with at least one enrolment.
- E2.3 **Joins vs. product.** Prove $r \bowtie_{\theta} s \equiv \sigma_{\theta}(r \times s)$. Give r, s where $r \times s \neq r \bowtie s$ and explain row counts.
- E2.4 **Division.** Let $\text{Takes}(\text{StudID}, \text{Code})$ and $\text{Required}(\text{Code})$ contain core courses. Write (i) $T \div \text{Required}$ in words, (ii) an equivalent expression without $\div$, (iii) why $\pi_{\text{StudID}}(T)$ is wrong.
- E2.5 **Equivalences.** Show $\sigma_p(r \cup s) \equiv \sigma_p(r) \cup \sigma_p(s)$ but $\pi_A(r \cup s) \not\equiv \pi_A(r) \cup \pi_A(s)$ in general? Actually prove or disprove the projection case; give a counterexample if it fails.

Chapter 3

SQL: Definition, Manipulation, and Querying

“SQL is the lingua franca of data.” Algebra is elegant on paper; SQL is how tables are actually created, filled, and queried in every DBMS from SQLite to BigQuery.

From zero: algebra $\rightarrow$ SQL. We assume only the relational model of Ch. 2 (tables, keys, $\sigma/\pi/\bowtie$). No prior SQL is assumed; every clause is introduced with an algebra gloss and a runnable example.

Learning Outcomes

After this chapter you will be able to:

- (L1) write DDL to create, alter, and drop tables with keys and checks;
- (L2) write DML to insert, update, and delete rows correctly;
- (L3) compose SELECT with WHERE, JOIN, GROUP BY, HAVING, ORDER BY, and subqueries;
- (L4) define and use views and explain updatability limits;
- (L5) translate between relational algebra and SQL and predict bag vs. set behaviour.

3.1 DDL: Creating the Schema

Data Definition Language declares the schema; the DBMS enforces it henceforth. Think “define the blueprint before laying bricks.”

Listing 3.1: DDL for NTU mini-schema

```
1 CREATE TABLE Student(  
2   id      CHAR(9) PRIMARY KEY,  
3   name    VARCHAR(64) NOT NULL,
```

```

4   major    VARCHAR(32) DEFAULT 'CS',
5   gpa      DECIMAL(3,2) CHECK (gpa BETWEEN 0.00 AND 5.00)
6 );
7 CREATE TABLE Course(
8   code     CHAR(6) PRIMARY KEY,
9   title    VARCHAR(64) NOT NULL,
10  credits  INT CHECK (credits IN (3,4))
11 );
12 CREATE TABLE Enrolled(
13   studID   CHAR(9) REFERENCES Student(id) ON DELETE CASCADE,
14   code     CHAR(6) REFERENCES Course(code) ON DELETE CASCADE,
15   grade    INT CHECK (grade BETWEEN 0 AND 100),
16   PRIMARY KEY (studID, code)
17 );

```

Key DDL elements: types (CHAR, VARCHAR, INT, DECIMAL, DATE), NOT NULL, DEFAULT, CHECK, PRIMARY KEY, FOREIGN KEY ...REFERENCES, UNIQUE. Actions on FK violation: CASCADE (propagate), SET NULL, RESTRICT/NO ACTION (reject), SET DEFAULT. ALTER TABLE adds/drops columns or constraints; DROP TABLE (with CASCADE) removes it.

Remark 3.1 (Domains in SQL). SQL domains are coarser than pure relational domains: CREATE DOMAIN GPA AS DECIMAL(3,2) CHECK(...) can reuse a constrained type across tables.

3.2 DML: Inserting, Updating, Deleting

Data Manipulation Language changes instances. Each statement is a transaction step (committed in Ch. 7).

Listing 3.2: DML examples

```

1  INSERT INTO Student VALUES ('U1234567A', 'Ann Tan', 'CS', 4.52);
2  INSERT INTO Enrolled(studID, code) VALUES ('U1234567A', 'SC2207'); -- grade NULL
3
4  UPDATE Student SET gpa = 4.65 WHERE id='U1234567A';
5  UPDATE Enrolled SET grade = 88
6  WHERE studID='U1234567A' AND code='SC2207';
7
8  DELETE FROM Enrolled WHERE grade < 50; -- withdraw fails
9  DELETE FROM Student WHERE id='U1234567A'; -- cascades to Enrolled

```

Constraints checked per row; the statement rolls back on violation. Bulk insert: INSERT INTO ...SELECT Upsert (PostgreSQL/MySQL): ON CONFLICT ...DO UPDATE. Beware bag semantics: inserting a duplicate PK fails, but inserting duplicate non-key rows succeeds (unlike set algebra).

3.3 SELECT: The Query Workhorse

Every SELECT maps to algebra: FROM ($\times/\bowtie$) $\rightarrow$ WHERE (σ) $\rightarrow$ GROUP BY + aggregation $\rightarrow$ HAVING (σ on groups) $\rightarrow$ SELECT (π) $\rightarrow$ ORDER BY $\rightarrow$ LIMIT.

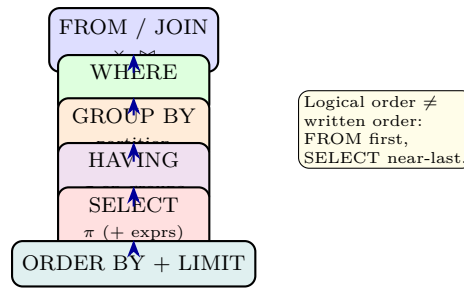


Figure 3.1: Logical evaluation order of SELECT. Optimisers reorder but preserve this semantics.

3.3.1 Filtering and Projection

Listing 3.3: WHERE and SELECT

```

1  -- Algebra:  $\pi_{\text{Name}}(\sigma_{\text{major}='CS'}(\text{Student}))$ 
2  SELECT DISTINCT name FROM Student WHERE major='CS';
3
4  -- Pattern, range, null
5  SELECT * FROM Student WHERE name LIKE 'Tan %' AND gpa BETWEEN 3.5 AND 5.0;
6  SELECT * FROM Enrolled WHERE grade IS NOT NULL;

```

DISTINCT de-duplicates to match set semantics; default ALL keeps bags (faster). LIKE with % (any) and _ (one char); BETWEEN, IN, IS NULL test null correctly (= NULL is always unknown).

3.3.2 JOINS

Listing 3.4: JOIN variants

```

1  -- Inner (natural on studID) vs explicit condition
2  SELECT S.name, E.grade
3     FROM Student S JOIN Enrolled E ON S.id = E.studID
4     WHERE E.code='SC2207';
5
6  -- Left outer: every student, even with no enrolment
7  SELECT S.name, E.code FROM Student S LEFT JOIN Enrolled E ON S.id=E.studID;
8
9  -- Self-join: pairs of students in same course
10 SELECT A.studID, B.studID
11     FROM Enrolled A JOIN Enrolled B
12     ON A.code=B.code AND A.studID < B.studID;

```

Types: [INNER] JOIN, LEFT/RIGHT/FULL OUTER JOIN, CROSS JOIN ($\times$), NATURAL JOIN. Outer joins pad with NULL. Join order matters for outer joins but not inner (optimizer picks cheapest).

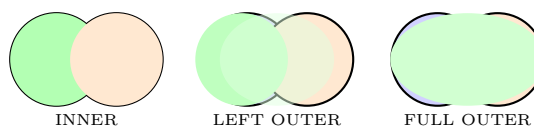


Figure 3.2: Join Venn intuition: inner keeps overlap only; left outer keeps all of left; full keeps all of both (null-padded).

3.3.3 Aggregation: GROUP BY and HAVING

Partitions rows by grouping columns, then aggregates each group.

Listing 3.5: Aggregation

```

1  -- Per-course enrolment count and average grade (NULLs ignored by AVG)
2  SELECT code, COUNT(*) AS n, AVG(grade) AS avg_g
3     FROM Enrolled GROUP BY code HAVING COUNT(*) >= 10;
4
5  -- Courses where every grade > 70? Use MIN
6  SELECT code FROM Enrolled GROUP BY code HAVING MIN(grade) > 70;
7
8  -- Overall stats vs per-group: WHERE before groups, HAVING after
9  SELECT major, COUNT(*) FROM Student WHERE gpa >= 3.0 GROUP BY major;

```

Aggregates: COUNT, SUM, AVG, MIN, MAX. Rule: every SELECT column must be aggregated or in GROUP BY (otherwise ambiguous). WHERE filters rows before grouping; HAVING filters groups after.

3.3.4 Subqueries and Set Operations

Listing 3.6: Subqueries and set ops

```

1  -- IN / EXISTS (EXISTS often fastest; IN handles NULL with care)
2  SELECT name FROM Student WHERE id IN (SELECT studID FROM Enrolled WHERE code='SC2207');
3  SELECT name FROM Student S WHERE EXISTS (SELECT 1 FROM Enrolled E WHERE E.studID=S.id AND
4     E.code='SC2207');
5
6  -- Scalar subquery in SELECT
7  SELECT name, (SELECT AVG(grade) FROM Enrolled WHERE studID=S.id) AS avg
8     FROM Student S;
9
10 -- UNION vs UNION ALL (ALL keeps duplicates, much cheaper)
11 SELECT id FROM Student WHERE major='CS'
12 UNION
13 SELECT studID FROM Enrolled WHERE code='SC2005';

```

Set ops (UNION, INTERSECT, EXCEPT) require union-compatible schemas and de-duplicate by default. Use ALL to keep bags.

3.4 Views

A *view* is a named, stored query—a virtual table.

Listing 3.7: Views

```

1  CREATE VIEW CS_Students AS SELECT * FROM Student WHERE major='CS';
2  CREATE VIEW CourseStats AS
3     SELECT code, COUNT(*) AS n, AVG(grade) AS avg_g FROM Enrolled GROUP BY code;
4
5  SELECT * FROM CS_Students WHERE gpa > 4.5; -- view expanded inline

```

```
6 DROP VIEW CourseStats;
```

Views provide logical independence and security (expose only needed columns). Materialized views (Ch. 6) store the result physically for speed. Updatability: a view is updatable if its definition is a single-table **SELECT** without aggregation, **DISTINCT**, **GROUP BY**, or joins; otherwise **INSTEAD OF** triggers are needed. **WITH CHECK OPTION** rejects inserts that would disappear from the view.

3.5 Programmatic Access (JDBC) and Authorization

Applications rarely type SQL by hand: *JDBC* (Java Database Connectivity) lets a program open a connection, send SQL, read results, and close—always close (prefer try-with-resources) or connections leak. Access control is orthogonal: **GRANT** gives a privilege on an object to a user or role, **REVOKE** takes it back. Rule of least privilege: grant only what the app needs.

Listing 3.8: JDBC query plus GRANT via Java

```
1 Connection c = DriverManager.getConnection("jdbc:postgresql://db/ntu", "app", "secret");
2 try (Statement s = c.createStatement()) {
3     s.executeUpdate("GRANT SELECT ON Student TO app_reader"); // authorize
4     ResultSet rs = s.executeQuery("SELECT name FROM Student WHERE major='CS'");
5     while (rs.next()) System.out.println(rs.getString("name"));
6 } finally { c.close(); } // always release
```

GRANT SELECT ON Student TO app_reader lets that role read one table only; **REVOKE SELECT ON Student FROM app_reader** undoes it. Views (Sec. 3.4) plus grants give column-level security: expose **CS_Students** instead of **Student**.

3.6 Chapter Notes

SQL standardised as SQL-86, now SQL:2023; every DBMS adds extensions. We used PostgreSQL-flavoured syntax; Oracle, MySQL, and SQL Server differ in types and outer-join syntax but share the core. Bag semantics (and its corner cases with **NULL**) is the price of performance.

3.7 Exercises

- E3.1 **Warm-up: DDL.** Add a **Prereq(code, prereq)** table with FKs to **Course** and a check preventing a course requiring itself. Why must the FK be composite or self-referencing?
- E3.2 **JOINS.** Write SQL for: (a) students who never enrolled, (b) courses with zero enrolments, (c) pairs of students sharing at least two courses. State which needs an outer join.
- E3.3 **Aggregation.** For **Enrolled**, write (a) per-student average, (b) majors whose average GPA exceeds 4.0, (c) courses where enrolment exceeds the overall average enrolment. Distinguish **WHERE** vs. **HAVING**.

- E3.4 **Subqueries vs. JOINS.** Rewrite an IN-subquery as a JOIN and as EXISTS; give data where IN returns empty due to NULL but JOIN/EXISTS do not. Which is safest?
- E3.5 **Views.** Create a view `GoodGrades( studID, avg)` for students with `AVG>80`. Is it updatable? What happens on `INSERT INTO GoodGrades...`? Propose a trigger-based fix.

Chapter 4

Relational Design and Normalisation

“A good schema tells no lies twice.” Normalisation is the systematic removal of redundancy and anomaly by decomposing tables according to the functional dependencies hidden in your data.

From zero: redundancy $\rightarrow$ dependencies. We start from an intuitive bad table (repeated facts, update pain) and derive functional dependencies, closures, and normal forms step by step. Only sets/relations from Ch. 2 assumed.

Learning Outcomes

After this chapter you will be able to:

- (L1) define functional dependencies and compute attribute closures;
- (L2) test membership via Armstrong’s axioms and find minimal covers;
- (L3) classify schemas as 1NF/2NF/3NF/BCNF and explain each violation;
- (L4) decompose losslessly and preserve dependencies (and know when both impossible);
- (L5) synthesise 3NF and decompose to BCNF algorithmically.

4.1 Good vs. Bad Design

Consider $R(\text{StudID}, \text{Code}, \text{StudName}, \text{CourseTitle}, \text{Instructor}, \text{Grade})$ where one row is one enrolment. StudName repeats for every course a student takes; CourseTitle repeats for every enrolled student. Problems:

- **Redundancy**—same fact stored many times (waste + risk).
- **Update anomaly**—change a course title in one row but miss another $\rightarrow$ inconsistency.
- **Insertion anomaly**—cannot insert a new course with no enrolments (PK needs StudID).

- **Deletion anomaly**—deleting the last enrolment of a course deletes the course itself.

Normalisation separates independent facts into their own tables, leaving each fact in exactly one place. In ER terms, each entity set and each M:N relationship becomes its own table with its own key—the mapping of Ch. 1 already guides you. FD analysis generalises that intuition to any set of constraints, even those not visible in the ER diagram.

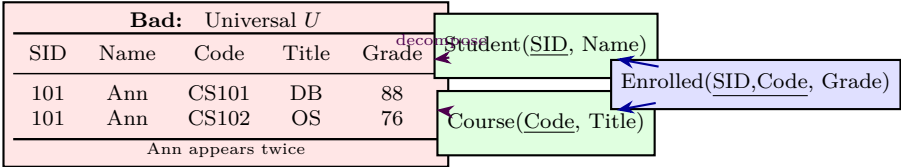


Figure 4.1: Bad universal table vs. 3NF decomposition. Each fact stored once; PKs/FKs preserve associations.

4.2 Functional Dependencies

We now state the central notion formally.

Definition 4.1 (Functional dependency). Let R be a schema. An FD $X \rightarrow Y$ (with $X, Y \subseteq R$) holds on instance r if for all tuples $t_1, t_2 \in r$, $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$. “ X determines Y .” A set F of FDs holds if every $f \in F$ holds.

Example: $\text{StudID} \rightarrow \text{StudName}$ (one name per ID), $\text{Code} \rightarrow \text{CourseTitle}$, $\{\text{StudID}, \text{Code}\} \rightarrow \text{Grade}$. Note $\text{StudName} \not\rightarrow \text{StudID}$ (names collide). FDs come from enterprise semantics, not from a sample instance—a single counterexample disproves an FD, but no instance proves one.

4.2.1 Closure and Armstrong’s Axioms

The *closure* X^+ is all attributes forced by X under F , computed by iterating: start with $X^+ := X$; repeatedly if $U \rightarrow V \in F$ and $U \subseteq X^+$ then $X^+ := X^+ \cup V$ until fixed.

Example 4.1 (Closure). $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow A\}$. Then $A^+ = \{A, B, C\}$ (via $A \rightarrow B$, then $B \rightarrow C$). CD^+ includes A via $CD \rightarrow A$, then B, C via $A \rightarrow B \rightarrow C$, so $CD^+ = \{A, B, C, D\}$ —a superkey.

Armstrong’s axioms are sound and complete: reflexivity ($Y \subseteq X \Rightarrow X \rightarrow Y$), augmentation ($X \rightarrow Y \Rightarrow XZ \rightarrow YZ$), transitivity ($X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$), from which union, decomposition, and pseudotransitivity follow. To test $F \models X \rightarrow Y$, check $Y \subseteq X^+$ (linear time). A *minimal cover* F_c is an equivalent, minimal set (singleton RHS, no redundant FD or attribute).

4.2.2 Keys from FDs

K is a *superkey* iff $K^+ = R$. It is a *candidate key* iff no proper subset has closure R . Enumerate candidate keys by testing closures of attribute subsets (prune supersets of found keys).

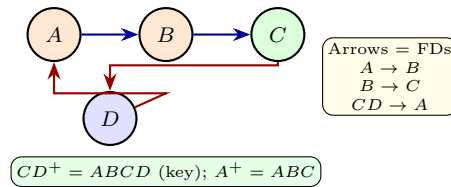


Figure 4.2: FD graph for $R(A, B, C, D)$. Closure follows arrows transitively; keys are attribute sets reaching all nodes.

4.3 Normal Forms

1NF Every attribute domain is atomic (no repeating groups, no nested tables). In SQL: no array-valued cells. Achieved by flattening.

2NF 1NF + every non-key attribute is *fully* dependent on *every* candidate key (no partial dependency on a proper part of a composite key).

3NF 2NF + no non-key attribute transitively depends on a key (if $X \rightarrow Y$ non-trivial, then X is a superkey or Y is prime—part of some key).

BCNF If $X \rightarrow Y$ non-trivial then X is a superkey. Strictly stronger than 3NF; every determinant must be a key.

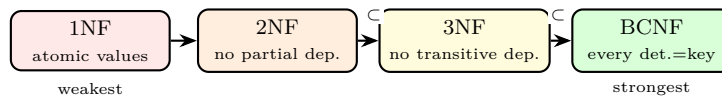


Figure 4.3: Normal-form hierarchy: each step eliminates a class of redundancy at the possible cost of dependency preservation (BCNF).

Example 4.2 (2NF violation). $R(\text{SID}, \text{Code}, \text{SName}, \text{Grade})$ with key $\{\text{SID}, \text{Code}\}$ and FD $\text{SID} \rightarrow \text{SName}$. SName depends on part of the key (SID)—partial dependency—so not 2NF. Decompose into $R_1(\text{SID}, \text{SName})$ and $R_2(\text{SID}, \text{Code}, \text{Grade})$.

Example 4.3 (3NF vs. BCNF). $R(\text{Course}, \text{Instructor}, \text{Room})$ with FDs $\text{Instructor} \rightarrow \text{Room}$, $\{\text{Course}, \text{Instructor}\}$ is key. $\text{Instructor} \rightarrow \text{Room}$ violates BCNF (Instructor not a superkey) but not 3NF if Room is prime? Here Room is not prime, so even 3NF fails. Decompose to $R_1(\text{Instructor}, \text{Room})$ and $R_2(\text{Course}, \text{Instructor})$.

A classic BCNF-but-not-dependency-preserving tension: $R(A, B, C)$ with $AB \rightarrow C$, $C \rightarrow B$. Candidate keys AB and AC ; $C \rightarrow B$ violates BCNF (C not superkey) but 3NF holds (B prime). BCNF decomposition loses $AB \rightarrow C$.

4.4 Decomposition: Lossless and Dependency-Preserving

A decomposition $\{R_1, \dots, R_k\}$ of R is:

- **Lossless** (non-additive) if $r = \pi_{R_1}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$ for every legal r —no spurious tuples. Binary test: $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$ (common attributes form a superkey of one piece).

- **Dependency-preserving** if every FD in F is implied by the union of projections $F_i = \{X \rightarrow Y \in F^+ \mid XY \subseteq R_i\}$ —checkable without joins.

3NF synthesis guarantees both. BCNF decomposition guarantees lossless but may sacrifice dependency preservation—the fundamental trade-off.

4.4.1 3NF Synthesis Algorithm

Given F , compute minimal cover F_c . For each $X \rightarrow Y \in F_c$, create $R_{XY} = XY$. If no R_i contains a candidate key of R , add a key relation. Merge duplicate schemas. The result is guaranteed lossless and dependency-preserving; if already 3NF, the algorithm may return the original schema unchanged.

Example 4.4 (Synthesis walk-through). $R(A, B, C, D, E)$ with $F = \{A \rightarrow B, C \rightarrow D, D \rightarrow E\}$. Minimal cover is itself. Schemas: $R_1(AB)$, $R_2(CD)$, $R_3(DE)$. No schema contains a key (AC is a key: $AC^+ = ABCDE$), so add $R_4(AC)$. Minimal grouping already; four tables suffice and preserve all FDs.

4.4.2 BCNF Decomposition Algorithm

While some R_i violates BCNF via $X \rightarrow Y$ (X not superkey in R_i): replace R_i by $R_{i1} = XY$ and $R_{i2} = R_i - Y$. Repeat; result is lossless and BCNF (but maybe not dependency-preserving). The choice of violating FD affects the final schema; heuristics prefer FDs with small XY to limit table count.

Listing 4.1: 3NF tables for enrolment

```

1 CREATE TABLE Student(sid CHAR(9) PRIMARY KEY, sname VARCHAR(64));
2 CREATE TABLE Course(code CHAR(6) PRIMARY KEY, title VARCHAR(64));
3 CREATE TABLE Enrolled(sid CHAR(9) REFERENCES Student,
4                       code CHAR(6) REFERENCES Course,
5                       grade INT, PRIMARY KEY(sid, code));

```

Remark 4.1 (Denormalisation). Real warehouses often store derived or repeated data (e.g. pre-joined Student+Enrolled) to avoid joins at query time. This is intentional denormalisation: accept controlled redundancy for read speed, guarded by ETL refresh or materialised views. OLTP stays normalised; OLAP may not.

4.4.3 Putting It Together: A Mini-Case

Start from a denormalised NTU table NTU_Record(SID, SName, Major, Code, Title, Instr, Room, Grade) with FDs $SID \rightarrow \{SName, Major\}$, $Code \rightarrow \{Title, Instr\}$, $Instr \rightarrow Room$, $\{SID, Code\} \rightarrow Grade$. Candidate key is $\{SID, Code\}$. This violates 2NF ($SID \rightarrow SName$ is partial) and 3NF ($Instr \rightarrow Room$ is transitive). 3NF synthesis yields Student(SID, SName, Major), Course(Code, Title, Instr), InstrRoom(Instr, Room), and Enrolled(SID, Code, Grade)—four tables, lossless and dependency-preserving. BCNF would further split Course if Title dependencies emerge. When in doubt, normalise first, measure, then consider denormalising only for hot read paths.

4.5 Chapter Notes

Normalisation theory was developed by Codd (1971–74), Boyce–Codd, and Fagin. Armstrong’s axioms and the chase test give the formal backbone. In practice: normalise to 3NF/BCNF for OLTP, then selectively denormalise for OLAP/read performance (Ch. 8). Higher NFs (4NF, 5NF) handle multi-valued and join dependencies. A useful habit: whenever you add a column, ask “what FD does it introduce and is its LHS a key?” If not, split.

4.6 Exercises

- E4.1 **Warm-up: closures.** $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$. Compute A^+ , AC^+ , all candidate keys. Is R in BCNF? Justify.
- E4.2 **Normal forms.** Universal $U(\text{SID}, \text{SIDName}, \text{Code}, \text{Title}, \text{Instr}, \text{Room})$ with $\text{SID} \rightarrow \text{SIDName}$, $\text{Code} \rightarrow \text{Title}$, $\text{Instr} \rightarrow \text{Room}$, key $\{\text{SID}, \text{Code}\}$. Identify 2NF and 3NF violations; propose a 3NF decomposition.
- E4.3 **Lossless test.** Decompose $R(A, B, C)$ with $A \rightarrow B$, $B \rightarrow C$ into $R_1(A, B)$, $R_2(B, C)$. Is it lossless? Dependency-preserving? Prove both with the tests above.
- E4.4 **BCNF vs. 3NF.** $R(A, B, C)$ with $AB \rightarrow C$, $C \rightarrow A$. Find keys, check 3NF and BCNF, then BCNF-decompose. Which FD is lost? Give an insertion that violates the lost FD but passes per-table checks.
- E4.5 **Synthesis.** Run 3NF synthesis on $R(A, B, C, D)$ with $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Write the resulting schemas, then reduce to minimal number of tables while staying 3NF.

Chapter 5

Storage, Files, and Index Structures

“A database lives on disk, accelerates in RAM, and earns its keep with indexes.” This chapter descends from abstraction to silicon: how tables become pages, files become heaps or sorted runs, and queries skip millions of rows via trees and hashes.

From zero: bits $\rightarrow$ B⁺ trees. We assume only the relational schema of Ch. 1–2. No OS/files prerequisite; disks, pages, and I/O cost are built from intuition with pictures before formalism.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain the memory hierarchy and why I/O cost dominates;
- (L2) describe heap, sorted, and hashed file organisations and their trade-offs;
- (L3) construct and search B⁺ trees and estimate their height/occupancy;
- (L4) compare B⁺ trees vs. hash indexes for point vs. range queries;
- (L5) choose an index type given a workload (equality, range, prefix).

5.1 The Memory Hierarchy and Disk

A program sees registers $\prec$ caches (ns) $\prec$ RAM (~ 100 ns) $\prec$ SSD ($\sim 100 \mu$ s) $\prec$ HDD (~ 10 ms) $\prec$ network. Databases care because *I/O dominates*: a single random HDD seek costs as much as millions of CPU ops. Optimising for I/O means grouping data into blocks and minimising the number of blocks read.

Definitions: *track*, *sector* (512 B–4 KB), *block/page* (DBMS unit, 4–16 KB), *seek + rotation + transfer* for HDD. On SSD, cost is erase-block and write amplification but still page-granular. Therefore the DBMS reads/writes *pages*, not bytes.

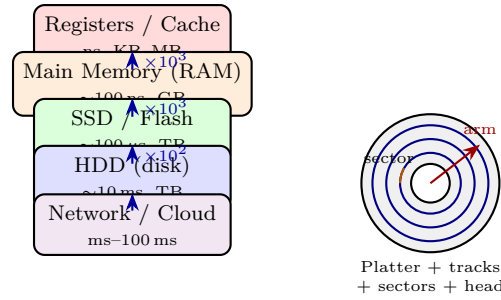


Figure 5.1: Memory hierarchy (latency grows $\sim 1000\times$ per level). Right: HDD anatomy; SSDs have no seek but still favour sequential page I/O.

5.1.1 Pages, Records, and Buffer Manager

Tables map to collections of pages on disk. Records (rows) are stored with a *page header* (slot directory) supporting variable-length fields and tombstones for deletions. The *buffer manager* caches pages in RAM frames, evicts via LRU/clock, and tracks dirty pages to write back. A query's I/O cost is the number of page reads/writes the buffer pool must service.

An *index* is an auxiliary structure mapping search keys to record locations (RIDs or primary keys) to avoid scanning the heap. Indexes live in pages too; choosing the right index (this section vs. next two) is physical design. The query optimiser (Ch. 6) decides automatically which index to use based on statistics and cost estimates.

5.2 File Organisations

- **Heap (unordered) file**—records appended wherever space exists; $O(B)$ scan, $O(1)$ insert (find free page), delete via free list or tombstone. Good for bulk scans if selection is not selective.
- **Sorted file**—records ordered by search key; binary search on pages $O(\log B)$ + sequential scan for range, but insert $O(B)$ to maintain order (use overflow chain or periodic reorganisation). External sort can rebuild sorted order offline.
- **Hashed file**—hash key $\rightarrow$ bucket page; $O(1)$ equality search on hash key, range queries impossible without full scan.
- **Clustered vs. unclustered**—clustered: table rows stored in key order (at most one per table); unclustered: index order differs from heap. Clustering accelerates range scans dramatically because qualifying rows are contiguous on disk.

Sequential I/O throughput is $10\text{--}100\times$ random I/O, so even $O(B)$ sequential scan can beat many random probes. Modern query execution (Ch. 6) therefore prefers sequential scans with prefetching when many pages are needed.

Remark 5.1 (Slotted pages). A page's slot directory holds offsets to each record, allowing variable-length rows and forwarding pointers after updates without moving RIDs. Free-space bitmaps accelerate insert page selection.

Example 5.1 (Cost comparison). Heap scan of $B = 5000$ pages: 5000 sequential I/Os (~ 50 ms on SSD). Sorted

file point lookup: $\log_2 B \approx 13$ random I/Os if binary searching pages, but with a B^+ index the same lookup is 3 I/Os. This order-of-magnitude gap motivates indexes even for modest tables.

5.3 B^+ Trees—The Default Index

A B^+ tree is a balanced, multi-way search tree whose leaves hold *all* data entries sorted and linked for range scans; internal nodes only guide search. Fan-out $f \sim 100\text{--}500$ (page size / entry size). Height $h = \lceil \log_f N \rceil$ is 2–4 for millions of rows, so point search costs h I/Os.

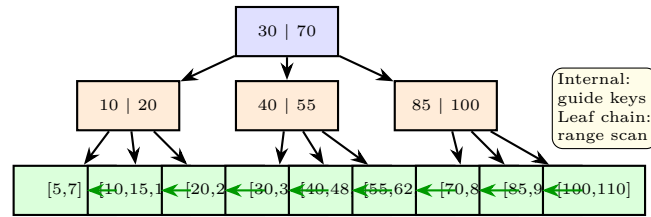


Figure 5.2: B^+ tree (order $m = 3$ shown small; real fan-out ~ 100 s). Search descends via guide keys; leaves linked for sequential range iteration.

Properties: all leaves at same depth, each node (except root) $\geq \lceil f/2 \rceil$ full (occupancy $\geq 50\%$), leaves doubly linked. Search: start at root, binary search guide keys, follow child pointer until leaf. Range $[L, U]$: find L , then walk leaf siblings until $> U$.

Operations: *insert* finds leaf, inserts sorted, splits leaf if overflow (push middle key up), propagating splits upward (height grows only at root). *Delete* removes, borrows or merges if underflow. Both $O(h)$ I/Os. Bulk load sorts first for $> 90\%$ occupancy.

Example 5.2 (Height estimate). $N = 10^7$ entries, page 16 KB, entry 32 B $\rightarrow f \approx 500$. $h = \lceil \log_{500} 10^7 \rceil = 3$. With root cached, point lookup = 2 I/Os. Doubling N rarely adds a level.

5.3.1 Clustered vs. Unclustered, Dense vs. Sparse

Dense: every search-key value appears; *sparse*: only some (e.g. one per page). Primary (clustered) B^+ tree is typically dense and clustered; secondary indexes are unclustered and must store record pointers (RIDs) or primary keys. Unclustered range scan may do one random I/O per row—potentially slower than a heap scan if many rows qualify.

5.4 Hash Indexes

For equality predicates (`WHERE id = ?`) hashing can be $O(1)$ average.

- **Static hashing**— $h(k) \rightarrow$ bucket page; overflow chain on collision; degrades with skew.
- **Extendible hashing**—directory doubles on bucket overflow; local/global depth track hashing bits; no overflow chain, directory fits in RAM.
- **Linear hashing**—splits buckets round-robin; no directory, graceful growth.

Hash indexes cannot answer $>$, **BETWEEN**, or prefix search (**LIKE** ‘CS%’) and are not order-preserving. Hence B^+ is the default; hash is an accelerator for point lookups (e.g. primary-key fetch). A covering hash that stores the whole tuple avoids the RID fetch but wastes space; most systems use B^+ even for equality because its $O(\log N)$ is close to $O(1)$ for realistic N and it handles all predicates.

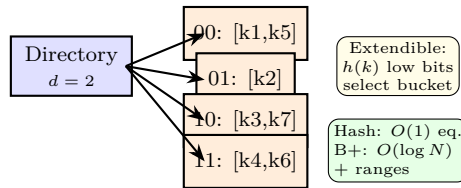


Figure 5.3: Extendible hashing directory and buckets. Hash excels at point lookup; B^+ at ordered access.

Choosing an index: cardinality, selectivity, and workload drive the decision. Low selectivity ($>10\%$ of rows) $\rightarrow$ heap scan may beat an unclustered index; prefix queries need B^+ ; high-cardinality equality on a small table may not justify any index (table scan cheaper than index + fetch). As a rule: index columns that appear in **WHERE** equality/range, **JOIN** keys, and **ORDER BY**; avoid over-indexing (each index slows inserts/updates, consumes space, and must be maintained under concurrency).

5.4.1 Example: Index Selection for NTU Schema

Consider queries: Q1 **WHERE** `sid='U1'` (point), Q2 **WHERE** `code='SC2207'` (moderate cardinality), Q3 **WHERE** `gpa BETWEEN 4.0 AND 5.0` (range). Suitable indexes: primary B^+ on Student.sid (clustered, for Q1), secondary B^+ on Enrolled.code or Course.code (for Q2), and B^+ on Student.gpa for Q3 (or a composite on (major,gpa) if major-filtered). Hash on sid would also serve Q1 but not Q3.

5.5 Chapter Notes

B-trees by Bayer–McCreight (1972); B^+ trees are their database refinement. Hashing variants from Fagin et al. (extendible) and Litwin (linear). Modern disks/SSDs shift constants but not the page-I/O model. LSM trees (log-structured merge) and column stores (Ch. 8) are optimised for write-heavy and analytic workloads, respectively. For deeper study, see Graefe’s “Modern B-tree Techniques” and the buffer-manager chapter of Ramakrishnan–Gehrke.

5.6 Exercises

E5.1 **Warm-up: hierarchy.** Estimate I/Os for a 10 M-row heap scan (page 16 KB, row 200 B) on HDD vs. SSD sequential vs. random. Why does sequential win?

E5.2 **B^+ height.** Page 8 KB, key+ptr 16 B, leaf occupancy 67%. Estimate fan-out, height for $N = 20M$, and I/Os for point and range-1000 queries.

E5.3 **Insert/split.** Insert keys 10,20,30,40,50 into an empty B^+ tree of order $m = 2$ (max 4 entries/leaf, min 2) step by step; show splits and resulting tree.

- E5.4 **Hash vs. B⁺** For workload: (a) `WHERE sid='U1'`, (b) `WHERE gpa BETWEEN 3.5 AND 4.0`, (c) `WHERE name LIKE 'Tan%'`, state which index (if any) helps and why; estimate I/Os with/without.
- E5.5 **Clustered choice.** A table has clustered B⁺ on (code) and unclustered on (gpa). A query filters `gpa>4.0` (5% rows) and projects code. Would an index-only scan on (gpa,code) help? Compare clustered vs. covering secondary.
- E5.6 **Composite prefix.** A B⁺ on (major,gpa) serves `WHERE major='CS' AND gpa>4.0` efficiently but not `WHERE gpa>4.0` alone. Explain via lexicographic order and prefix matching.

Chapter 6

Query Processing, Optimisation, and Cost Estimation

“Declarative in, procedural out—cost decides.” You write what you want; the DBMS figures out how to get it fast by enumerating plans and picking the cheapest using statistics.

From zero: algebra $\rightarrow$ execution. We assume Ch. 2–3 (algebra, SQL, indexes) but no systems knowledge. Every operator and cost formula is motivated with a picture before symbols used.

Learning Outcomes

After this chapter you will be able to:

- (L1) outline the lifecycle $\text{parse} \rightarrow \text{rewrite} \rightarrow \text{optimise} \rightarrow \text{execute}$;
- (L2) describe physical operators for selection, projection, join, and aggregation;
- (L3) estimate I/O and CPU cost using histograms and cardinalities;
- (L4) compare join algorithms (nested-loop, hash, sort-merge) by cost;
- (L5) read an EXPLAIN plan and spot a bad cardinality estimate.

6.1 From Declarative Query to Execution Plan

Parsing turns SQL into a logical algebra tree $(\sigma, \pi, \bowtie)$. The rewriter applies equivalences: push selections past joins/projections, eliminate redundant subqueries, and flatten views. The optimiser then translates each logical operator to one or more physical operators and chooses the combination with lowest estimated cost. Finally the executor runs the chosen plan using an iterator (volcano) model: each operator implements **open-next-close** and pulls tuples on demand.

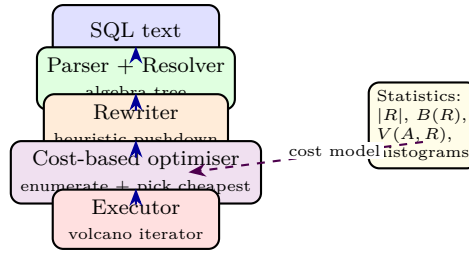


Figure 6.1: Query compilation pipeline. Rewriter applies algebra laws (Ch. 2); optimiser uses statistics to cost physical plans.

6.1.1 Logical vs. Physical Operators

Logical $\bowtie$ might become physical *hash join*, *sort-merge join*, or *nested-loop join* depending on indexes, sizes, and predicates. Logical σ might be an index scan vs. heap scan. This decoupling lets the optimiser explore dozens to thousands of plans for a single query.

6.2 Physical Operators and Their Costs

We count page I/Os plus CPU per tuple; buffer hits are cheaper than misses but we estimate worst-case I/Os first. Let $B(R)$ be pages in R , $|R|$ tuples, M buffer frames.

Selection $\sigma_{A=c}(R)$. Heap scan $B(R)$ I/Os. B^+ on A : $h + \lceil \text{sel} \cdot |R| / \text{rec/page} \rceil$ (clustered) or $h + \text{sel} \cdot |R|$ random for unclustered (may be worse than scan if selectivity high). Use index when selectivity low.

Projection π with duplicate elimination. Sort-based $O(B \log B)$ or hash-based $O(B)$; if an index already orders by the projection keys, a scan suffices. SQL without **DISTINCT** skips elimination.

Aggregation and GROUP BY. Sort or hash partitioning; similar cost to projection.

JOIN is the expensive part. Three families:

- **Block nested-loop (BNL):** for each block of outer R scan inner S . Cost $\approx B(R) + |R_{\text{blocks}}| \cdot B(S)$. Good when S small or no index.
- **Index nested-loop:** for each $r \in R$ probe index on S . Cost $|R| \cdot (h + 1)$ if index on join key. Wins when R small and S indexed.
- **Hash join (Grace):** partition both on hash of join key, then probe. Cost $\approx 3(B(R) + B(S))$ (partition+probe), linear. Best for large equi-joins without index.
- **Sort-merge join:** sort both (if not already sorted/index-ordered) then merge. Cost $\text{sort} + B(R) + B(S)$. Cheapest when inputs already sorted or output must be sorted.

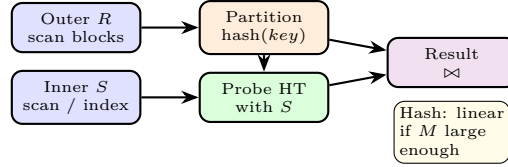


Figure 6.2: Hash join dataflow. Partition phase splits both relations; probe phase emits matches without sorting.

Example 6.1 (Choosing a join). $|R| = 10k$, $|S| = 1M$, S has B^+ on join key, $M = 1000$. Index nested-loop: $10k \text{ probes} \times 3 \text{ I/Os} = 30k$. Hash join: $3(1k+10k) \approx 33k$. BNL without index: far worse. Index NL wins.

6.2.1 Other Physical Choices

Sequential scan with prefetch vs. *index scan*; *covering index* (index-only scan) avoids heap fetch; *bitmap* intersection for multi-column ANDs. Sort for ORDER BY may reuse a merge-join’s order.

Example 6.2 (Covering index). Query `SELECT code, avg(grade) FROM Enrolled WHERE code='SC2207' GROUP BY code` needs only `code` and `grade`. A B^+ on $(code, grade)$ answers it from the index alone (index-only scan), saving the heap fetch per row—often $2\times$ faster.

6.2.2 Pipelining vs. Materialisation

Pipelined execution streams tuples operator-to-operator without writing intermediates to disk (volcano pulls). Blocking operators (SORT, HASH BUILD) must materialise before producing output. Pipelining saves I/Os but limits parallelism; optimisers weigh this via cost models with CPU and memory terms.

6.3 Cardinality Estimation and Histograms

Cost needs cardinalities. The catalog stores per table: $|R|$, $B(R)$, distinct values $V(A, R)$, min/max, and per column a *histogram* (equi-width or equi-depth) and most-common-values list.

Estimate selectivity *sel*:

$$\begin{aligned}
 |\sigma_{A=c}(R)| &\approx |R|/V(A, R), \\
 |\sigma_{A>c}(R)| &\approx |R| \cdot (\max - c)/(\max - \min), \\
 |R \bowtie_A S| &\approx |R| \cdot |S|/\max(V(A, R), V(A, S))
 \end{aligned}$$

assuming uniform distribution and inclusion. Histograms refine range estimates: sum buckets up to c . Errors compound across joins—a bad histogram can mispredict by orders of magnitude, leading the optimiser to pick nested-loop over hash join catastrophically.

Keeping stats fresh (ANALYZE, auto-vacuum) and using extended stats (functional-dependency, n-distinct for multi-columns) are the cheapest optimisation you can do. When the optimiser is wrong, materialised CTEs or hints can pin a plan.

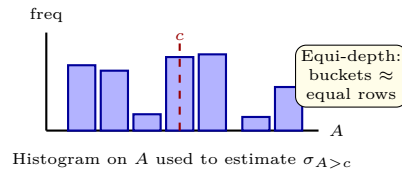


Figure 6.3: Histogram buckets give piecewise-uniform estimate. Equi-depth handles skew better than equi-width.

Remark 6.1 (Correlated columns). Independence assumption fails when columns correlate (e.g. `major` and `gpa` if CS curves differ). Multi-column histograms or learned estimators fix this; without them, $|\sigma_{\text{major}=\text{CS} \wedge \text{gpa}>4.0}|$ estimated as product of selectivities underestimates truth.

Example 6.3 (Estimation error propagation). Three joins each mispredicted by $3\times$ yield $27\times$ total error. The optimiser may then choose a nested-loop that runs for minutes where a hash join finishes in seconds. EXPLAIN ANALYZE exposes the divergence row-by-row.

6.4 Plan Enumeration and EXPLAIN

For n joins, bushy vs. left-deep search spaces explode. Most optimisers use dynamic programming over subsets ($O(3^n)$) for $n \leq 10$ – 12 and greedy/heuristic beyond. They cost each subplan with the formulas above and keep the cheapest per interesting order.

Listing 6.1: Reading EXPLAIN

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT S.name, AVG(E.grade) FROM Student S
3     JOIN Enrolled E ON S.id=E.studID
4     WHERE S.major='CS' GROUP BY S.name;
5 -- Seq Scan on Student (cost=... rows=500 sel=0.2)
6 --   -> Hash Join (cost=... rows=8000)
7 --       -> Index Scan on Enrolled(code)

```

Look for: estimated rows vs. actual rows (ANALYZE), unexpected Seq Scans where an index exists, nested-loop with large outer, and spilled sorts (`work_mem` exceeded). If estimated rows $\ll$ actual, the histogram is stale.

Remark 6.2 (Parametric and adaptive). Prepared statements reuse a plan for many parameter values; a plan optimal for selective c may be terrible for unselective c (parameter sniffing). Adaptive execution (re-optimize mid-flight, e.g. Spark AQE) mitigates this.

6.4.1 Heuristic Rewriting First

Before costing, the rewriter applies cheap logical wins: push selections past joins (reduce input size early), push projections (trim width), eliminate redundant distinct, and unnest subqueries to joins where equivalent. These heuristics are always beneficial and shrink the DP search space, leaving the cost-based stage to choose among already-sensible plans.

Example 6.4 (Unnesting). `SELECT * FROM R WHERE R.a IN (SELECT S.a FROM S WHERE S.b=5)` rewrites to $R \bowtie_{\sigma_{b=5}}(S)$ when $S.a$ is a key, enabling a hash join instead of a correlated nested loop—often $10\times$ faster.

Remark 6.3 (Interesting orders). If a later operator needs sorted output (`ORDER BY`, `GROUP BY`, merge join), preserving order early can avoid a final sort. Optimisers keep the cheapest plan *per order* (“interesting orders”) to capture this trade-off, not just the globally cheapest.

6.5 Chapter Notes

System R (Selinger et al., 1979) pioneered cost-based optimisation with DP and the formulas above; Volcano/Cascades generalised enumeration. Reading: Garcia-Molina Ch. 16, Neumann et al. on adaptive joins. Modern cloud optimisers add learned cardinalities and materialised-view rewriting.

6.6 Exercises

- E6.1 **Warm-up: costs.** $B(R) = 1000$, $B(S) = 500$, $V(A, R) = 100$, $V(A, S) = 500$. Estimate $|\sigma_{A=5}(R)|$ and $|R \bowtie_A S|$. Recalculate if A uniform vs. histogram showing $A = 5$ is $5\times$ average frequency.
- E6.2 **Join choice.** Using BNL, index-NL ($h = 3$), and Grace hash costs, pick the cheapest for $|R| = 50k$ ($B = 500$), $|S| = 500k$ ($B = 5000$), index on $S.A$ present, $M = 100$. Show working.
- E6.3 **Pushdown.** Prove $\sigma_{p(R)}(R \bowtie S) \equiv \sigma_p(R) \bowtie S$ when p mentions only R . Draw the rewriter’s transformation and explain why pushed plans are usually cheaper.
- E6.4 **Histograms.** Construct equi-depth (3 buckets) for column values $[1, 1, 1, 2, 5, 5, 9, 20]$. Estimate count for `WHERE A BETWEEN 2 AND 6` via histogram vs. uniform; which is closer to truth?
- E6.5 **EXPLAIN drill.** Given a plan showing Hash Join with estimated rows $10\times$ actual and a 2 GB spill, propose two fixes (stats, memory, or rewrite) and predict each fix’s cost effect.
- E6.6 **Interesting orders.** Query `SELECT * FROM R JOIN S ON R.a=S.a ORDER BY R.a` may benefit from sort-merge join even if hash join is slightly cheaper. Explain why preserving sort order can dominate the trade-off.

Chapter 7

Transactions, Concurrency Control, and Recovery

“ACID is a contract: either everything happens, or nothing does—and no one sees the middle.”
Concurrency makes databases fast; transactions make that speed correct.

From zero: interleavings to isolation. We assume only SQL/relations from Ch. 3; no OS concurrency assumed. Every anomaly is shown by a timeline first, then formalised.

Learning Outcomes

After this chapter you will be able to:

- (L1) state ACID and explain each property’s implementation sketch;
- (L2) enumerate dirty read / non-repeatable read / phantom and lost update;
- (L3) test serialisability via precedence graphs and explain 2PL;
- (L4) distinguish 2PL, strict 2PL, and their deadlock handling;
- (L5) outline WAL, checkpointing, and recovery (undo/redo).

7.1 Transactions and ACID

A *transaction* is a logical unit of work (e.g. transfer \$100 from A to B: read A, $A:=A-100$, write A, read B, $B:=B+100$, write B) that the DBMS executes as if it were alone and atomic.

Atomicity all-or-nothing: commit makes all effects permanent; abort rolls back none (via undo log).

Consistency preserves integrity constraints (keys, checks); transactions are responsible for correctness, the DBMS enforces it.

Isolation concurrent transactions appear serial; the I in ACID (the hard one).

Durability committed effects survive crashes (via write-ahead logging to stable storage).

SQL: BEGIN (or implicit), COMMIT, ROLLBACK, savepoints.

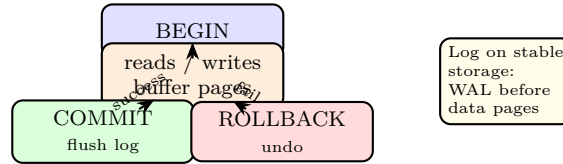


Figure 7.1: Transaction lifecycle. WAL ensures commit durability and abort atomicity even across crashes.

7.2 Why Concurrency Is Hard: Anomalies

Interleaving without isolation breaks intuition.

- **Dirty read**—T2 reads T1’s uncommitted write; T1 aborts → T2 saw data that never existed.
- **Lost update**—T1 and T2 read x , both add 1, both write; one increment lost.
- **Non-repeatable read**—T1 reads x , T2 commits new x , T1 reads again and gets different value.
- **Phantom**—T1 scans “all CS students”, T2 inserts a new CS student, T1 repeats scan and finds a new row (predicate changed).

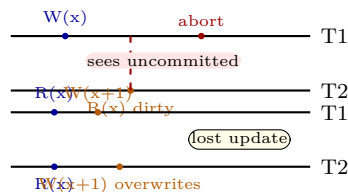


Figure 7.2: Dirty read (top) and lost update (bottom) timelines. Distance = time; arrows mark reads/writes.

A *schedule* is an interleaving of operations $r_i(x), w_i(x), c_i, a_i$. It is *serial* if transactions do not interleave; *serialisable* if equivalent to some serial order (correctness criterion). *Recoverable* adds: no commit of T2 that read dirty from T1 before T1 commits/aborts; *cascadeless* = no dirty reads at all. *Strict* = no read or write of uncommitted data (both reads and writes wait).

7.2.1 Precedence (Serialisation) Graph

Nodes = transactions; edge $T_i \rightarrow T_j$ if they conflict (r/w or w/w on same item and at least one is w) and T_i ’s operation precedes T_j ’s. Theorem: schedule is conflict-serialisable iff graph is acyclic; any topological order is an equivalent serial schedule. Conflict serialisability is stricter than view serialisability (which allows blind writes to reorder) but efficiently testable.

Example 7.1 (Graph test). $S : r_1(x) w_2(x) r_1(y) w_2(y)$ has edges $T_1 \rightarrow T_2$ (on x) and $T_2 \rightarrow T_1$? No, $r_1(y)$ before $w_2(y)$ gives $T_1 \rightarrow T_2$ only—acyclic $\Rightarrow$ serialisable as $T_1; T_2$. Adding $w_2(x)$ before $r_1(x)$ would flip one

edge and may create a cycle.

Remark 7.1 (View vs. conflict). Example $w_1(x)w_2(x)w_3(x)$ is view-serialisable (final write decides) but not conflict-serialisable under our test—the graph has edges $T_1 \rightarrow T_2 \rightarrow T_3$ (acyclic) actually both pass here; a true separating example needs blind writes interleaved with reads. In practice conflict serialisability is the enforced criterion.

7.3 Locking and Two-Phase Locking (2PL)

Pessimistic concurrency: lock before accessing, unlock after.

Shared S (read) vs. eXclusive X (write) locks: S - S compatible, all else conflicts. Protocol: well-formed (lock before op, unlock after) plus *two-phase*: no lock after first unlock (growing $\rightarrow$ shrinking). This guarantees serialisability (but not freedom from deadlock or starvation).

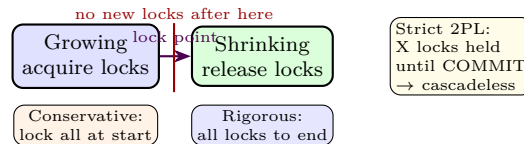


Figure 7.3: Two-phase locking phases. Strict 2PL (hold X to commit) gives recoverability and is used in practice.

Variants: *strict 2PL* holds X locks to commit/abort (ensures cascadeless + strict execution); *rigorous 2PL* holds all locks to end; *conservative* predeclares read/write sets. Strict 2PL is the practical default (e.g. PostgreSQL’s row locks).

Isolation levels in SQL map to lock strictness: **READ UNCOMMITTED** (no read locks), **READ COMMITTED** (short read locks), **REPEATABLE READ** (strict 2PL on accessed rows), **SERIALIZABLE** (strict 2PL + predicate/gap locks for phantoms). Weaker levels allow some anomalies for throughput.

7.3.1 Deadlock

Two transactions wait for each other’s locks. Detection via waits-for graph cycle; resolution by aborting a victim (fewest locks, youngest). Prevention: wait-die / wound-wait (timestamp ordering), or timeout. Deadlock rate grows with contention; keeping transactions short is the best antidote.

7.3.2 Beyond Locking: MVCC and Optimistic

Multi-version concurrency control (MVCC) keeps old row versions so readers never block writers (each read sees a snapshot as of its start time). Writes still lock. PostgreSQL, Oracle, MySQL/InnoDB use MVCC for high read concurrency. *Optimistic* (validation) protocols let transactions run unsynchronised and check for conflicts at commit (good for low contention). Timestamp ordering assigns each T a timestamp $TS(T)$ and aborts operations that would violate timestamp order—an alternative to locking.

Example 7.2 (MVCC snapshot). T_1 at time 10 writes $x := 5$. T_2 starts at time 12 and reads x —it sees 5 even if T_1 has not committed, but its own write $x := 7$ creates a new version invisible to T_2 ’s snapshot. Commit order determines which version survives.

7.4 Recovery: WAL and Checkpoints

Disks may crash mid-write. *Write-ahead logging* (WAL) rule: log record describing change reaches stable storage *before* the data page does. Each log entry: $\langle T, X, \text{old}, \text{new} \rangle$, LSN, prev-LSN chain, plus $\langle T \text{ COMMIT} \rangle$ / $\langle T \text{ ABORT} \rangle$.

On crash, ARIES-style recovery does three passes: *Analysis* (who was active, dirty pages), *Redo* (replay all logged changes to bring DB to crash state), *Undo* (roll back loser transactions using old values, writing compensating records). Checkpoints (`BEGIN_CKPT`, `END_CKPT` with active-T table and dirty-page table) bound redo work.



Figure 7.4: ARIES recovery passes. Redo is forward from last checkpoint; undo walks loser chains backward.

Example 7.3 (WAL correctness). T updates $x : 10 \rightarrow 20$. Log $\langle T, x, 10, 20 \rangle$ flushed, then page flushed, then $\langle T \text{ COMMIT} \rangle$ flushed. Crash after page flush but before commit: redo reapplies $10 \rightarrow 20$, then undo rolls back $20 \rightarrow 10$ because no commit—atomicity preserved.

Steal/no-force policy: buffer manager may write uncommitted pages (steal $\rightarrow$ need undo) and need not force pages at commit (no-force $\rightarrow$ need redo). WAL supports both.

Remark 7.2 (Idempotence and compensation). Redo is idempotent (repeating does not change result) because LSNs guard double application. Undo writes *compensation log records* (CLRs) so a crash during undo can itself be recovered by resuming undo from CLRs.

7.5 Chapter Notes

Serialisability theory from Papadimitriou (1979), Bernstein et al. 2PL from Eswaran et al. (1976). ARIES (Mohan et al., 1992) is the recovery gold standard. MVCC theory in Bernstein–Goodman (1983). For practice: run concurrent `psql` sessions with `BEGIN ISOLATION LEVEL SERIALIZABLE` and observe serialization failures.

7.6 Exercises

- E7.1 **Warm-up: ACID.** Map each of Atomicity/Consistency/Isolation/Durability to the mechanism that implements it (WAL, locks, constraints, log flush). What breaks if WAL is disabled?
- E7.2 **Anomalies.** Give schedules demonstrating each of dirty read, lost update, non-repeatable read, phantom. For each, state the weakest isolation level that forbids it.
- E7.3 **Serialisability.** Schedule $r_1(x)r_2(x)w_1(x)r_2(y)w_2(y)w_1(y)$. Draw its precedence graph, test serialisability, and if serialisable give an equivalent serial order.
- E7.4 **2PL.** Show a 2PL schedule that is not strict and exhibits a dirty read. Then make it strict by holding X locks; explain why cascading abort disappears.

E7.5 **Recovery.** Log: $\langle T1, x, 10, 20 \rangle \langle T2, y, 5, 15 \rangle \langle T1 \text{ COMMIT} \rangle \langle \text{crash} \rangle$. With WAL and steal/no-force, which passes (redo/undo) touch which records? What are final x, y after recovery?

Chapter 8

Advanced Topics: NoSQL, NewSQL, and Distributed Databases

“One size does not fit all.” The relational engine optimised for transactions now shares the stage with key-value stores, documents, graphs, columns, and distributed SQL that trades consistency for scale.

From zero: scale $\rightarrow$ trade-offs. We assume Ch. 1–7 (relations, algebra, storage, transactions). No distributed-systems prerequisite; CAP, replication, and sharding are built from intuition with pictures.
--

Learning Outcomes

After this chapter you will be able to:

- (L1) compare NoSQL data models (key-value, document, column, graph) on use cases;
- (L2) explain CAP and PACELC and where real systems sit;
- (L3) contrast replication (single-leader, multi-leader, leaderless) and quorum;
- (L4) describe sharding strategies and their routing costs;
- (L5) evaluate NewSQL vs. NoSQL vs. traditional RDBMS for a workload.

8.1 Beyond Relations: Why NoSQL?

Relational OLTP shines for structured data with ACID. But web-scale workloads brought: massive write throughput, flexible schemas (each user profile differs), semi-structured JSON/XML, graph traversals (friends-of-friends), and time-series columns where column-stores beat row-stores 10 $\times$.

- **Scale-out**—commodity servers, not bigger single boxes.

- **Schema flexibility**—add fields without ALTER TABLE locks.
- **Specialised shape**—graphs and wide columns are awkward as joins.

NoSQL = “Not Only SQL”—many systems still offer SQL-like queries. Trade-off: weaker transactional guarantees (BASE: Basically Available, Soft state, Eventually consistent) for horizontal scalability and lower latency.

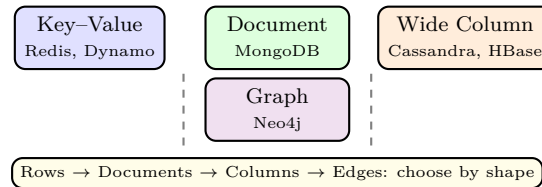


Figure 8.1: NoSQL model spectrum. Key-value is fastest for point lookups; documents allow nested JSON; columns compress time series; graphs optimise traversals.

8.1.1 The Four NoSQL Families

Key-Value. Map *key* → *blob*. $O(1)$ get/put, no query language beyond key; perfect for sessions, caches, shopping carts. Example: GET `user:1001` → JSON blob.

Document. Key → JSON/BSON document with indexed fields. Query by field, secondary indexes, aggregation pipelines. Flexible schema; joins are application-side or via \$lookup (costly). Example: MongoDB `db.users.find({major:"CS", gpa: {$gt:4.0}})`.

Wide Column / Column-Family. Row key → column families (each sorted map column→value). Physically column-oriented (Ch. 5’s column store) → excellent compression and scan of few columns over many rows (time series, sensor data). LSM-based.

Graph. Nodes + labelled edges with properties; query language Cypher/Gremlin traverses paths (`MATCH (a:User)-[:FOLLOWS]->(b)`). Adjacency lists indexed; 2-hop query that would be two self-joins in relational becomes a pointer walk.

8.1.2 When to Stay Relational?

If you need multi-row ACID transactions, foreign keys, and ad-hoc joins with consistency, relational (or NewSQL) remains superior. Many architectures are *polyglot*: RDBMS for transactions, Redis for cache, Mongo for catalogs, Neo4j for recommendations.

Example 8.1 (Polyglot at NTU). NTU CourseReg RDBMS handles Enrolled with ACID enrolment transactions. Redis caches hot student profiles ($O(1)$ session lookup). Elasticsearch (document-ish) powers course search with full-text ranking. A nightly ETL feeds a column store (BigQuery) for analytics—four stores, each optimal for its shape, synced via CDC (change-data capture) logs.

Remark 8.1 (Convergence). Modern systems blur the lines: PostgreSQL stores JSONB, supports graph via AGE, and has columnar extensions; MongoDB adds multi-document ACID; Cassandra adds lightweight transactions. The spectrum matters more than the label.

8.2 CAP, PACELC, and Consistency Models

Definition 8.1 (CAP). In a distributed data store subject to a network *partition* P , you can have at most one of *Consistency* C (linearizability: every read sees the latest write) and *Availability* A (every request gets a non-error response). Formalised by Gilbert–Lynch (2002, 2012).

During a partition, CP systems refuse or block (e.g. traditional RDBMS, HBase, ZooKeeper); AP systems serve stale data (e.g. Cassandra, Dynamo). After the partition heals, AP systems reconcile via eventual consistency. PACELC extends: if Partition, trade A vs. C ; Else, trade Latency vs. C .

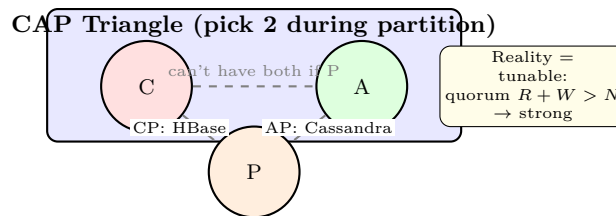


Figure 8.2: CAP is ternary only during partitions; otherwise latency/consistency is tunable via quorums.

Consistency spectrum: *linearizable* (single-copy illusion, expensive) $\succ$ *sequential* $\succ$ *causal* $\succ$ *eventual* (reads converge eventually, cheapest). *Eventual* plus *read-your-writes* and *monotonic reads* often suffices for user-facing apps (your own post appears immediately, others may see it slightly later).

8.3 Replication and Quorums

Replication provides availability and fault tolerance.

- **Single-leader** (primary–replica)—writes to leader, async/sync replication to followers; reads from followers trade recency for load.
- **Multi-leader**—any datacentre’s leader accepts writes; async reconciliation (conflict-free replicated data types or last-write-wins).
- **Leaderless** (Dynamo-style)—any replica accepts read/write; quorum $R + W > N$ ensures overlap and thus recency (e.g. $N = 3$, $W = 2$, $R = 2$). Hinted handoff and anti-entropy repair handle failures.

Synchronous replication waits for follower ack (CP, higher latency); *asynchronous* acks immediately (AP, risk of loss on leader crash). Semi-sync (ack from at least one follower) is the common compromise.

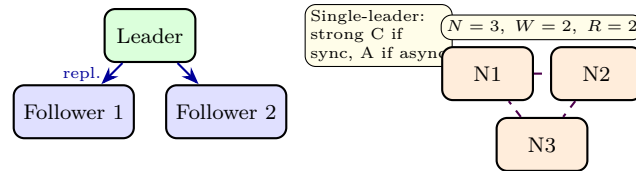


Figure 8.3: Left: single-leader replication. Right: leaderless quorum ring; write touches W , read touches R , overlap guarantees recency when $R + W > N$.

8.4 Sharding (Partitioning)

A table too large for one node is split across shards by a *partition key*:

- **Range**—contiguous key ranges per shard; good for range queries but hot-spots if writes monotonic (timestamps).
- **Hash**— $h(key) \bmod N$ spreads uniformly; range queries scatter to all shards (scatter-gather).
- **Consistent hashing**—ring with virtual nodes; adding a node moves only $1/N$ of keys; used by Dynamo/Cassandra.

Routing via coordinator or client-side shard map; scatter-gather queries pay fan-out latency; cross-shard transactions need two-phase commit (expensive) so design hot transactions to be single-shard (choose partition key = tenant ID / user ID that groups co-accessed data).

8.4.1 Example: Sharding NTU Enrolled

Partition Enrolled by $\text{hash}(\text{studID})$ —all of a student’s enrolments co-locate, so “transcript for student X” is single-shard. A query “all students in SC2207” scatters. If that query is frequent, secondary index or dual write to a course-sharded copy helps.

Remark 8.2 (Shard size and rebalancing). Ideal shard holds 10–100 GB so that rebalancing (splitting or moving a shard) completes in minutes, not hours. Too many tiny shards inflate routing metadata; too few huge shards make recovery slow. Consistent hashing with virtual nodes gives fine-grained rebalancing without global hash recomputation.

8.5 NewSQL: ACID at Scale

NewSQL (Spanner, CockroachDB, TiDB, VoltDB) gives RDBMS semantics (SQL, ACID, joins) with NoSQL-like horizontal scale. Techniques: deterministic concurrency (VoltDB), TrueTime/clock-bounded waits (Spanner), Raft/Paxos-replicated shards with distributed strict 2PL or OCC, cost-based distributed optimiser aware of shard locality. When your workload is relational but outgrows a single primary, evaluate NewSQL before hand-sharding. Cloud offerings (Spanner, Cockroach Cloud, Yugabyte) manage replication and rebalancing automatically; you still choose partition keys and isolation levels wisely.

The decision tree: start relational; if write QPS $> 10\text{k/s}$ or data $> \text{TB}$ and sharding becomes painful, benchmark a NewSQL candidate; if data is inherently non-relational (JSON, graph) or needs sub-ms KV, add a

NoSQL sidecar rather than forcing relations.

Remark 8.3 (Spanner’s TrueTime). Spanner bounds clock uncertainty (~ 7 ms) and waits out uncertainty before committing, giving external consistency (linearizability) without global locks. CockroachDB uses hybrid logical clocks with similar guarantees but without atomic clocks—cheaper, slightly higher tail latency.

<i>Remark 8.4</i> (Choosing a stack).	Workload	Candidates	Must-have	Avoid
	OLTP (banking)	RDBMS / NewSQL	ACID, FKs	eventual
	Session / cache	Redis (KV)	latency	graph
	Catalog (JSON)	Mongo (doc)	flex schema	wide-col
	Time series	Cassandra	col scan	KV
	Recommendations	Neo4j	traversals	hash sharded RDBMS

8.6 Chapter Notes

NoSQL survey: Cattell (2011), Sadalage–Fowler. CAP/PACELC: Brewer (2000), Gilbert–Lynch, Abadi. Replication/sharding: Kleppmann’s *Designing Data-Intensive Applications* (2017) is the accessible bible. NewSQL: Stonebraker, Spanner (Corbett et al., OSDI’12), CockroachDB. Try Cockroach Labs’ interactive CAP demo. For hands-on, spin up a 3-node Cassandra vs. a single Postgres and measure $p95$ latency under partition (via `iptables DROP`) to feel CAP in practice.

8.7 Exercises

- E8.1 **Warm-up: models.** For each of (a) shopping cart, (b) student JSON profiles with varying fields, (c) “friends of friends” query, (d) IoT sensor timeseries, pick KV / document / graph / wide-column and justify via access pattern.
- E8.2 **CAP.** An e-commerce cart uses Dynamo ($N = 3, W = 2, R = 2$). Show a partition where a write succeeds on one side and a read on the other returns stale data if $R + W \leq N$. Fix W, R for strong consistency.
- E8.3 **Quorums.** With $N = 5$, what minimal W, R give linearizable reads/writes with best availability? What if you optimise for read-heavy (small R)? Compute fault tolerance ($N - W$ writes survive, $N - R$ reads survive).
- E8.4 **Sharding.** Enrolled hash-partitioned by studID has hot-spot on Hash(SC2207)? Explain why range vs. hash matters and propose a partition key that keeps both “per-student” and “per-course” queries local—or argue why impossible and propose a secondary index strategy.
- E8.5 **Polyglot design.** Design a polyglot stack for NTU’s learning platform (grades need ACID, lecture videos need CDN, social graph needs recommendations, analytics needs warehouse). Assign each component to a store family and state consistency requirements.
- E8.6 **NewSQL vs. sharded RDBMS.** Compare Spanner’s TrueTime commit wait vs. hand-sharded MySQL

with 2PC for a cross-shard transfer. Which has lower tail latency and why? When would you still choose hand-sharding?